ESPECIFICACIÓN DE PROMPTS PARA IA

Proyecto Aplicaciones de Biometría y Medio Ambiente

Imanol Figuero Parras **28/09/2026**

SECCIÓN 1: PROMPT PARA GENERACIÓN DE LA BASE DE DATOS

Por favor, actúa como un Ingeniero de Bases de Datos Senior. Necesito que generes el script SQL completo DDL y DML para la base de datos del proyecto de monitorización ambiental/biométrica.

1. DISEÑO FORMAL EN NUESTRA NOTACIÓN (v3) [ENTIDADES Y TIPOS ABSTRACTOS]:
   - Entidad Usuarios: (id_usuario : Z, nombre: Text, password_hash : Text, email: Text, fecha_registro : Z).
   - Entidad Mediciones: (id_medicion : Z, dispositivo_id : Text, tipo_medicion : Z, valor: R, contador: N, timestamp : Z) Donde tipo_medicion sigue la enumeración: MedicionesID = {CO2 = 11, TEMPERATURA = 12, RUIDO = 13}.

2. ACLARACIONES Y ESTRUCTURA DEL DISEÑO:
   - La tabla 'Usuarios' gestionará la autenticación de usuarios del sistema.
   - La tabla 'Mediciones' almacenará la telemetría enviada por los sensores IoT/Beacons.
   - Debe existir una clave foránea opcional entre Mediciones y Usuarios si la medición está asociada a un usuario específico.

3. DIRECTRICES EXACTAS DE IMPLEMENTACIÓN:
   - Motor objetivo: MySQL 8.0 / MariaDB.
   - Usa tipos nativos correctos: Z -> INT/BIGINT, R -> DOUBLE/FLOAT, N -> INT UNSIGNED, Text -> VARCHAR/TEXT.

- Incluye restricciones de integridad: PRIMARY KEY, UNIQUE, FOREIGN KEY con ON DELETE CASCADE/SET NULL, NOT NULL donde corresponda.
- Agrega índices para optimizar consultas por dispositivo_id, tipo_medicion y timestamp.
- Incluye un script de inserción (Seeds) con al menos 3 usuarios de prueba y 10 mediciones simuladas de CO2 y Temperatura.

4. REQUISITOS DE COMENTARIOS Y DOCUMENTACIÓN:
   - Incluye comentarios al inicio de cada tabla especificando su firma matemática en notación formal v3.
   - Agrega comentarios en el código SQL explicando cada restricción e índice creado.

5. TESTS AUTOMÁTICOS:
   - Genera un script SQL de prueba (o test de integración en PHPUnit/Python) que verifique:
     a) La inserción correcta de registros.
     b) El fallo esperado al violar una restricción UNIQUE o NOT NULL.
     c) La integridad referencial al eliminar un usuario asociando sus mediciones.

SECCIÓN 2: PROMPT PARA LA LÓGICA DE NEGOCIO REAL EN BACKEND (PHP)

Actúa como Arquitecto de Software PHP Senior. Diseña e implementa la capa de Lógica de Negocio Pura (dominio) del backend en PHP 8.x.

1. DISEÑO FORMAL EN NUESTRA NOTACIÓN:

--------- LogicaNegocio ---------
|
| (Funciones puras de dominio sin estado HTTP)
|
|

usuario: Text --> diHola() --x

res: (nombre: Text, saludo: Text) <--

|
| |

nombre: Text, --> hacerLogin() --x

password: Text

autenticado: B <--

|
| |

disp: Text, --> guardarMedicion() --x

tipo: Z, val: R, cont: N

exito: B <--

|
| |

disp: Text, --> obtenerUltimasMediciones() --x

limite: N

res: [(tipo: Z, valor: R, timestamp: Z)] <-- |

---

2. ACLARACIONES Y ESTRUCTURA DEL DISEÑO:
   - Esta capa es LÓGICA PURA DE DOMINIO: totalmente desacoplada de la capa de transporte (sin $_GET, $_POST, $_SESSION, ni llamadas a echo/header).
   - Interactúa con la base de datos usando PDO con sentencias preparadas (Prepared Statements) para prevenir inyecciones SQL.
   - El login debe verificar contraseñas encriptadas usando password_verify().

3. DIRECTRICES EXACTAS DE IMPLEMENTACIÓN:
   - Crea la clase/módulo 'LogicaNegocio.php' en PHP 8.2+ con tipado estricto (declare(strict_types=1);). - Devuelve estructuras DTO o arrays asociativos fuertemente tipados.
   - Devuelve estructuras DTO o arrays asociativos fuertemente tipados.
   - Maneja excepciones personalizadas (ej. DomainException, AuthenticationException).

4. REQUISITOS DE COMENTARIOS Y DOCUMENTACIÓN:
   - En la cabecera PHPDoc de CADA función, DEBES incluir la firma matemática exacta en notación v3: Ejemplo:
     /**
     * Firma formal v3: nombre: Text, password: Text --> hacerLogin() --x --> autenticado: B

* Valida las credenciales de usuario contra la BD.
*/

5. TESTS AUTOMÁTICOS:
   Genera la suite completa de Tests Unitarios con PHPUnit (LogicaNegocioTest.php) utilizando Mocks de PDO para probar:
   a) Login exitoso con credenciales válidas.
   b) Login fallido con contraseña incorrecta o usuario inexistente.
   c) Guardado de mediciones dentro de rangos válidos.
   d) Manejo de excepciones ante fallos de conexión a BD.

SECCIÓN 3: PROMPT PARA EL SERVIDOR REST EN BACKEND (PHP)

Actúa como Desarrollador de APIs RESTful en PHP. Necesito el código de la capa de controladores/endpoints REST que expone la Lógica de Negocio mediante HTTP JSON.

1. DISEÑO FORMAL EN NUESTRA NOTACIÓN:



res: RespuestaREST <--

|

Tipos compuestos requeridos:

- RespuestaSaludo = (nombre: Text, saludo: Text, error: Text).
- RespuestaLogin = (resultado: B, usuario: Text, error: Text).
- RespuestaREST = (codigo : Z, cuerpo: Text).

2. ACLARACIONES Y ESTRUCTURA :
   - Los controladores reciben peticiones HTTP POST con JSON en el cuerpo.
   - Gestionan la sesión PHP de forma idempotente con session_status() === PHP_SESSION_NONE.
   - Invocan a la capa 'LogicaNegocio' y traducen las respuestas a JSON estandarizado.

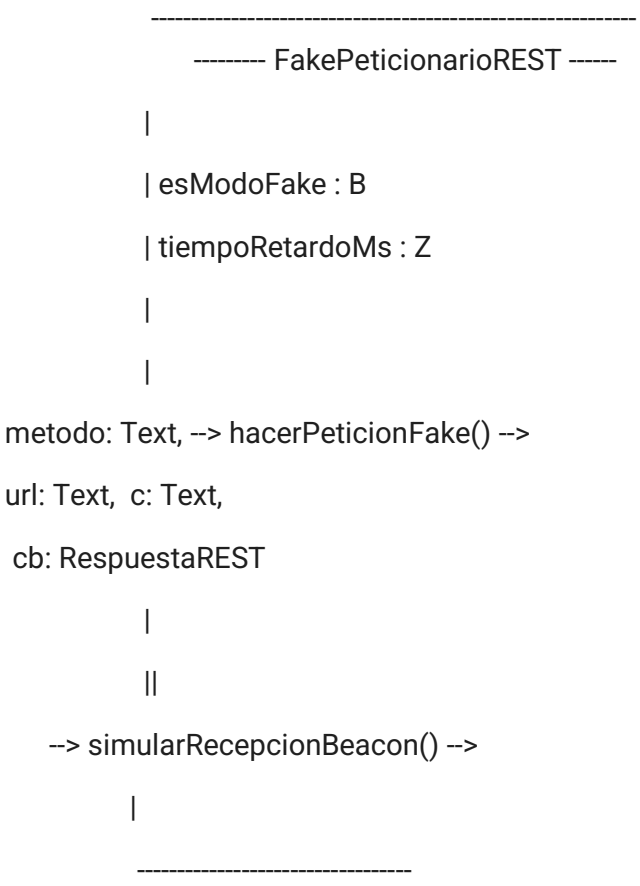
3. DIRECTRICES EXACTAS DE IMPLEMENTACIÓN:
   - Formato de entrada: Lee el cuerpo con file_get_contents('php://input') y decodifica el JSON.
   - Cabeceras HTTP obligatorias: header('Content-Type: application/json; charset=utf-8');. - Manejo de códigos de estado HTTP semánticos: 200 OK, 400 Bad Request, 401 Unauthorized, 500 Internal Error.
   - Manejo de códigos de estado HTTP semánticos: 200 OK, 400 Bad Request, 401 Unauthorized, 500 Internal Error.
   - Garantiza que los campos de respuesta conserven tipos estables tanto en éxito como en error (ej. campo 'error' siempre como Text).

4. REQUISITOS DE COMENTARIOS Y DOCUMENTACIÓN:
   En la cabecera PHPDoc de CADA función, DEBES incluir la firma matemática exacta en notación v3: Ejemplo:
   - Cada archivo/endpoint debe comenzar con la firma formal v3 en el bloque de comentarios.
   - Breves comentarios explicando la gestión de sesiones y lectura de buffers HTTP.
5. TESTS AUTOMÁTICOS:
   Genera un script de test de integración con cURL (o una colección ejecutable de Pest / Postman) que verifique:
   a) POST a /rest/hacerLogin.php con JSON válido -> 200 OK y Cookie de Sesión.
   b) Intento de acceso a endpoint protegido sin sesión -> 401 Unauthorized.
   c) Envío de JSON malformado -> 400 Bad Request.

SECCIÓN 4: PROMPT PARA LA LÓGICA FAKE EN EL TELÉFONO (ANDROID JAVA)

Actúa como Ingeniero Android Senior. Diseña un servicio 'Fake' en Java para probar el cliente móvil sin depender del servidor REST ni del hardware BLE real.

1. DISEÑO FORMAL EN NUESTRA NOTACIÓN:



2. ACLARACIONES Y ESTRUCTURA:
   - La clase simula la red móvil introduciendo un retardo artificial (ej. 500 ms) usando Handler o ExecutorServices.
   - Si la URL solicitada es 'hacerLogin', devuelve una respuesta simulada JSON de login exitoso o fallido según los parámetros.
   - Genera tramas iBeacon aleatorias o fijas para simular la detección de telemetría CO2 (11) y Temperatura (12).

3. DIRECTRICES EXACTAS DE IMPLEMENTACIÓN:

- Lenguaje: Java 11 / Android SDK.
- Debe implementar la misma interfaz o firmas que la clase 'PeticionarioREST' real para permitir conmutación transparente mediante una constante booleana (USA_LOGICA_FAKE = true).
- No debe realizar conexiones de red reales (ninguna llamada a HttpURLConnection).

4. REQUISITOS DE COMENTARIOS Y DOCUMENTACIÓN:
   - Incluye en la cabecera de la clase y de cada método la firma matemática en notación formal v3.
   - Comentarios explicativos sobre la generación del JSON simular y retardo asíncrono.
5. TESTS AUTOMÁTICOS:
   Escribe una suite de Tests Unitarios en Android (con JUnit 4/5 y Robolectric) que compruebe:
   a) Que la llamada a hacerPeticionFake() ejecuta el callback asíncrono en menos de 1000 ms.
   b) Que el JSON generado en el mock contiene los campos requeridos (codigo: Z, cuerpo: Text).

SECCIÓN 5: PROMPT PARA LA LÓGICA FAKE EN EL NAVEGADOR (JAVASCRIPT)

Actúa como Desarrollador Frontend Senior. Crea un módulo JavaScript ES6 que simule la lógica del cliente y del servidor REST directamente en el navegador web.

1. DISEÑO FORMAL EN NUESTRA NOTACIÓN:

-------- FakeUX_LogicaCliente ------

|

| simuladorActivo : B

| latenciaSimuladaMs : Z

|

|

cb: (err: Text, res: RespuestaSaludo) ------> diHola() -->

|

||

nom: Text, pass: Text, cb: RespLogin -------> hacerLogin() -->

|

|

||

cb: (res: [Medicion]) ----------------------> obtenerMedicionesFake() -->

|

---

2. ACLARACIONES Y ESTRUCTURA:
   - El módulo intercepta o reemplaza las llamadas AJAX reales (XMLHttpRequest/Fetch) por respuestas simuladas mediante Promises y setTimeout.
   - Almacena el estado de sesión y usuarios de prueba dentro de 'localStorage' para mantener la persistencia durante la sesión de navegación.

3. DIRECTRICES EXACTAS DE IMPLEMENTACIÓN:
   - Código JavaScript moderno (ES6 Modules, async/await, Promises).
   - Genera datos aleatorios coherentes de telemetría (ej. CO2 entre 400 y 1200 ppm, Temperatura entre 18.0 y 30.0 °C).
   - Proporciona un método de conmutación activarFake(modo: B).

4. REQUISITOS DE COMENTARIOS Y DOCUMENTACIÓN:
   - Cabecera JSDoc en cada función especificando la firma formal v3:
     /**
     * @signature nom: Text, pass: Text, cb: RespLogin --> hacerLogin() -->
     */

5. TESTS AUTOMÁTICOS:
   Crea un archivo de tests automáticos con Jest o Vitest para verificar:
   a) La resolución correcta de las Promises en diHola() y hacerLogin().
   b) La estructura correcta de los objetos JSON devueltos.
   c) La actualización simulada de localStorage tras un login.

SECCIÓN 6: PROMPT PARA LA UX E INTERFAZ GRÁFICA EN EL NAVEGADOR

Actúa como Diseñador y Desarrollador Frontend UI/UX. Crea la interfaz de usuario web (Dashboard y Login) responsiva y moderna para la monitorización biométrica y ambiental.

1. DISEÑO GRÁFICO Y ARQUITECTURA DE PANTALLA:
   - Header / Barra superior: Logo del proyecto ("Biometría & IoT"), indicador de estado de conexión (verde/rojo) y nombre del usuario autenticado con botón 'Cerrar Sesión'.
   - Sección 1 - Formulario de Login:
     - Tarjeta centrada con campos: 'Usuario' (text), 'Contraseña' (password) y botón 'Iniciar Sesión'.
     - Mensaje flotante de error/éxito.
   - Sección 2 - Dashboard de Telemetría (Grid Responsivo):
     - Tarjeta KPI 1: Nivel de CO2 actual (ppm) con cambio de color según umbral (<800 ppm Verde, 800-1000 Amarillo, >1000 Rojo).
     - Tarjeta KPI 2: Temperatura actual (°C).
     - Gráfico temporal dinámico (Línea de evolución) para mostrar las últimas mediciones.

2. DISEÑO FORMAL EN NUESTRA NOTACIÓN:

-------- UX_InterfazWeb ------

|

| estaAutenticado: B

| datosMediciones: [Medicion]

|

|

renderizarLogin() -->

|

||

usr: Text --> renderizarDashboard() -->

|

|

||

tipo: Z, val: R--> actualizarIndicadorKPI() -->

|

---

3. DIRECTRICES EXACTAS DE IMPLEMENTACIÓN:
   - Tecnologías: HTML5 semántico, CSS3 puro (Flexbox/Grid, CSS Variables, modo oscuro elegante), JavaScript Vanilla.
   - Para el gráfico de evolución, usa la librería Chart.js o SVG nativo.
   - Totalmente responsive (adaptable a móvil, tablet y escritorio).
   - Integración limpia con los módulos 'UX_LogicaCliente' (tanto en modo REAL como FAKE).

4. REQUISITOS DE COMENTARIOS:
   - Comentarios en HTML/CSS estructurando cada sección.
   - Firmas formales v3 en las funciones JavaScript de control de la UI.

5. TESTS AUTOMÁTICOS:
   Escribe un script de prueba E2E con Cypress o Playwright que compruebe:
   a) La visibilidad del formulario de login al cargar la página.
   b) El flujo de login completando los campos y verificando que se muestra el Dashboard.
   c) La actualización de los valores de las tarjetas KPI tras recibir nuevas mediciones.